

CampusBite --- Complete DBMS Project Guide

1. Project Overview

CampusBite is a campus food-ordering platform for VIT Chennai. Students, teachers and other authorized users can discover food outlets located in different campus areas, browse menus, place orders for pickup, make payments, and review completed orders. Shop owners and staff can manage menus and process incoming orders.

The project is particularly suitable for a **DBMS-focused academic project** because it naturally demonstrates:

- Entity--Relationship modeling
- Relational schema design
- Primary and foreign keys
- 1:1, 1:N and M:N relationships
- Associative/junction tables
- Self-referencing relationships
- Normalization up to 3NF/BCNF where appropriate
- Candidate keys and alternate keys
- Domain, entity and referential integrity
- CHECK, UNIQUE and NOT NULL constraints
- Transactions and ACID properties
- Concurrency control
- Indexing and query optimization
- Views
- Stored procedures and functions
- Triggers
- Audit/history tables
- Aggregation, joins, subqueries and CTEs
- Window functions
- Role-based access control
- SQL security
- Backup/recovery concepts
- Temporal/history data
- Reporting and analytics

The goal should not merely be to make a website that works. The project should demonstrate **why the database is designed the way it is and how DBMS features solve real problems in the application.**

2. Recommended Technology Stack

Core Stack

Layer Recommended technology Why

Frontend React + TypeScript Component-based UI, strong ecosystem, type safety

Backend Node.js + Express + Simple REST API, fast TypeScript development, easy SQL integration

Database MySQL 8.0+ Excellent relational DBMS for an academic DBMS project

SQL access `mysql2` / Lets you explicitly parameterized SQL demonstrate SQL rather than hiding everything behind an ORM

Authentication JWT + bcrypt/Argon2 Secure login/session architecture

API testing Postman or Bruno Easy demonstration of REST APIs

API documentation Swagger/OpenAPI Makes backend easy to demonstrate

Version control Git + GitHub Tracks schema and application changes

Containerization Docker + Docker Compose Reproducible MySQL/backend environment

DB migration Flyway or Liquibase Version-controlled database schema changes

Optional cache Redis Useful for menu/cache experiments, but not required for the first version

Why MySQL?

MySQL is a strong choice for this project because it provides:

- Primary and foreign keys
- Transactions

- InnoDB
- CHECK constraints
- UNIQUE constraints
- Indexes
- Views
- Stored procedures
- Stored functions
- Triggers
- CTEs
- Window functions
- EXPLAIN/EXPLAIN ANALYZE
- Transaction isolation
- Referential integrity

It is also widely understood, easy to install, and straightforward to demonstrate during a DBMS viva.

Why avoid making an ORM the center of the project?

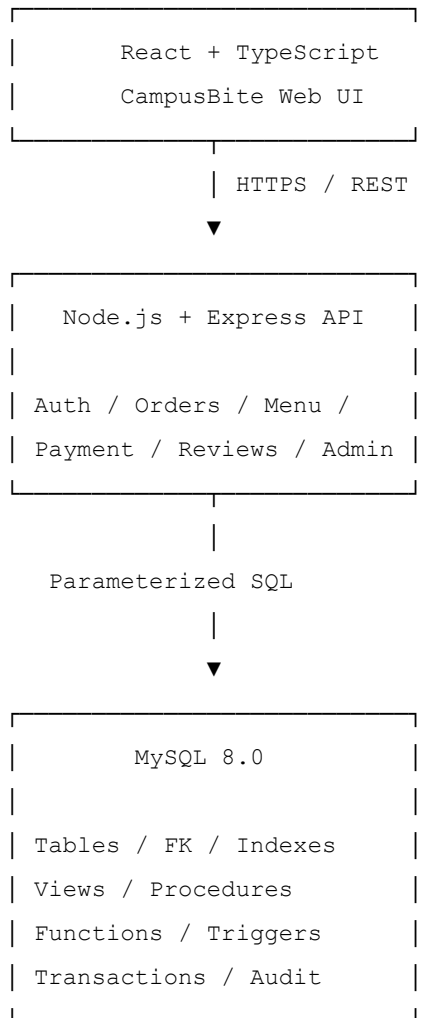
ORMs such as Prisma, Sequelize and TypeORM can be useful in production, but for a DBMS subject they can hide the SQL concepts you want your evaluator to see.

For CampusBite, a good academic approach is:

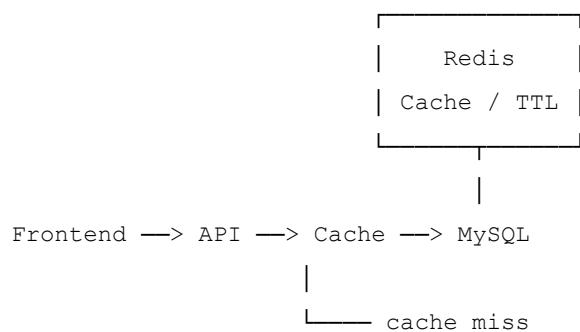
Application code + parameterized handwritten SQL + selected stored procedures/views/triggers

You can still use a lightweight data-access layer, but important queries should remain visible in SQL.

3. High-Level Architecture



Optional:



For the first implementation, Redis is optional. **Do not add infrastructure simply to make the project look complex.**
Add it only if you demonstrate why it is needed.

4. Main Actors

4.1 Customer

A customer can be a:

- Student
- Teacher

Customer operations:

1. Register/login
 2. View campus areas
 3. View shops
 4. View shop timings
 5. Browse food categories
 6. Browse menu
 7. View item availability
 8. Add items to cart
 9. Place order
 10. Choose payment method
 11. Track order status
 12. View order history
 13. Leave a review after completion
-

4.2 Shop Staff

Staff members work for one or more shops.

Typical workflow:

1. Login
2. View incoming orders
3. Accept/start preparation
4. Change order to `preparing`
5. Change order to `ready`
6. Hand over order
7. Mark order `completed`
8. View previous orders

A staff member should not be able to process arbitrary orders belonging to unrelated shops.

That rule should be enforced by **application authorization plus database design/queries**, not merely by hiding buttons in the frontend.

4.3 Shop Owner

An owner can:

- Create/manage shop information
 - Manage food items
 - Change prices
 - Enable/disable items
 - Manage shop staff
 - Configure timings
 - View sales reports
 - View customer reviews
 - View inventory-related information if inventory is added
-

4.4 Administrator

A future administrator role could:

- Manage users
 - Approve shops
 - Disable shops
 - Manage categories
 - Resolve disputes
 - View system-wide analytics
 - Review audit logs
-

5. Real-World User Case Study

Student Scenario

Suppose a student wants food from a shop near North Square.

Step 1 --- Login

The student enters email and password.

The backend:

1. Finds the user by indexed email.
2. Verifies the password hash.
3. Creates an authenticated session/token.

Relevant DB concept:

- UNIQUE constraint
- Index
- Authentication
- Parameterized query

Step 2 --- Discover Shops

The student selects:

North Square

The application queries:

```
SELECT shop_id, name, status
FROM Shop
WHERE area_id = ?
ORDER BY name;
```

DB concepts:

- Foreign key
- Selection
- Index
- JOIN if area information is displayed

Step 3 --- View Menu

The student selects a shop.

The application retrieves available items:

```
SELECT
    fi.item_id,
    fi.name,
    c.name AS category,
    fi.price,
    fi.description
FROM Food_Item fi
JOIN Category c
    ON fi.category_id = c.category_id
WHERE fi.shop_id = ?
    AND fi.is_available = TRUE
ORDER BY c.name, fi.name;
```

DB concepts:

- INNER JOIN
- Filtering
- Indexing
- Normalized category data

Step 4 --- Place Order

Suppose the student orders:

- 1 sandwich
- 2 juices

The backend should use a **transaction**.

Conceptually:

BEGIN

1. Validate customer
2. Validate shop
3. Validate every item
4. Verify items belong to the selected shop
5. Verify items are currently available
6. Read current prices
7. Insert Orders row
8. Insert Order_Items rows
9. Calculate/verify total
10. Create Payment row

COMMIT

If any important operation fails:

ROLLBACK

This demonstrates **atomicity**.

Step 5 --- Staff Processes Order

The staff dashboard shows:

Order #1052

Sandwich × 1

Juice × 2

Total: ₹180

Status: PLACED

Staff changes:

PLACED



PREPARING



READY



COMPLETED

Every transition can be recorded in:

Order_Status_History

This creates an audit trail.

Step 6 --- Payment

Payment has a lifecycle:

PENDING



SUCCESS

or:

PENDING



FAILED

or, for a successful payment later reversed:

SUCCESS



REFUNDED

The payment record should contain a transaction/reference ID supplied by the payment system.

Step 7 --- Review

After the order becomes completed, the customer can submit:

Rating: 5

The database should prevent reviews for orders that do not belong to the customer.

This is a good example of a **business rule that must be enforced deliberately**.

6. Current Schema Analysis

Your current design already contains several good DBMS ideas:

- Users
- Student/teacher distinction
- Areas
- Shops
- Shop staff
- Categories
- Food items
- Shop timings
- Orders
- Order items
- Payments
- Price snapshot at order time
- Foreign keys
- Composite primary key in Shop_Staff

However, the ER diagram you shared contains some entities that are not present in the SQL you supplied, notably:

- Student_Profile
- Teacher_Profile
- Order_Status_History
- Review

The diagram and SQL should eventually represent **one consistent schema**.

7. Important Schema Improvements

7.1 Separate Authentication/User Identity from Role/Profile

Currently:

```
Users
├─ user_type
├─ role
└─ reg_no
```

This can become difficult because:

```
user_type = student
role = owner
```

is technically possible even if that combination is not logically meaningful.

A cleaner design is:

```
Users
├─
├─ Student_Profile
└─ Teacher_Profile
```

Then:

Student_Profile

```
user_id PK/FK
reg_no UNIQUE
hostel_area_id FK
```

Teacher_Profile

```
user_id PK/FK
employee_id UNIQUE
```

This is a classic example of **specialization/generalization** in an ER model.

8. Role Design Improvement

If users can have multiple roles, using:

```
role ENUM('customer','staff','owner')
```

is restrictive.

A more extensible design is:

```
Users
|
| M:N
|
User_Roles
|
Roles
```

For example:

```
Roles
-----
role_id
role_name
```

```
User_Roles
-----
user_id
role_id
```

Then one person can be:

```
customer
+
staff
```

without modifying the Users table.

For a smaller academic project, the ENUM approach is acceptable, but the M:N role model is more extensible and demonstrates another relational concept.

9. Category Hierarchy --- An Excellent DBMS Innovation

Your ER diagram shows:

```
Category
|
└─ parent_category_id
```

This is a **self-referencing relationship**.

For example:

```
Food
├─ Fast Food
│   └─ Burgers
│       └─ Sandwiches
├─ Beverages
│   └─ Cold Drinks
│       └─ Hot Drinks
└─ South Indian
```

This is valuable for demonstrating recursive relationships.

Example query:

```
SELECT
    child.name AS category,
    parent.name AS parent_category
FROM Category child
LEFT JOIN Category parent
    ON child.parent_category_id = parent.category_id;
```

You can also use a recursive CTE to display an entire category tree.

10. Order_Items as an Associative Entity

An order can contain many food items.

A food item can occur in many orders.

Therefore:

```
Orders M:N Food_Item
```

cannot be directly represented in a relational table.

You resolve it using:

```
Order_Items
```

So:

```
Orders 1 — N Order_Items N — 1 Food_Item
```

This is one of the most important relational modeling concepts in the project.

11. Why price_at_order Is Correct

Your:

```
price_at_order DECIMAL(8,2)
```

is an excellent design decision.

Suppose:

```
Burger today = ₹80
```

Customer orders it.

Later:

```
Burger = ₹100
```

If Order_Items only stored:

```
item_id
```

then historical order displays could incorrectly show ₹100.

By storing:

```
price_at_order = 80
```

the historical transaction remains correct.

This is a controlled form of **historical snapshot data**.

A useful explanation in viva:

The current Food_Item price represents the current catalog state, while price_at_order represents the price that was legally/business-wise applicable to a particular transaction.

12. Order Status History

Add:

```
CREATE TABLE Order_Status_History (  
    history_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    order_id INT NOT NULL,  
    changed_by INT NULL,  
    old_status VARCHAR(30),  
    new_status VARCHAR(30) NOT NULL,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (changed_by) REFERENCES Users(user_id)  
);
```

Then:

```
Order 1052
```

```
10:01 placed
```

```
10:03 preparing
```

```
10:17 ready
```

```
10:22 completed
```

This gives you:

- Auditing
- Temporal history
- Accountability
- Reporting
- Trigger opportunities

13. Review Table

A suitable review table:

```
CREATE TABLE Review (  
    review_id INT PRIMARY KEY AUTO_INCREMENT,  
    order_id INT NOT NULL UNIQUE,  
    customer_id INT NOT NULL,  
    shop_id INT NOT NULL,  
    rating TINYINT NOT NULL,  
    comment VARCHAR(500),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (customer_id) REFERENCES Users(user_id),  
    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id),  
  
    CHECK (rating BETWEEN 1 AND 5)  
);
```

However, there is an important integrity rule:

customer_id and shop_id should correspond to the customer and shop of the referenced order.

That is a good opportunity to discuss **composite constraints/triggers/application validation**.

14. Timing Improvements

Your Timing table should allow multiple periods per day if required.

For example:

```
Monday
11:00-15:00
18:00-21:00
```

A simple:

```
shop_id + day_of_week
```

UNIQUE constraint would prevent that.

If you want multiple intervals, keep:

```
timing_id
shop_id
day_of_week
open_time
close_time
```

and allow multiple rows.

You should also validate:

```
open_time < close_time
```

with a CHECK constraint if overnight hours are not supported.

If you want shops open across midnight, model that explicitly rather than using an ambiguous time rule.

15. Database Normalization

15.1 First Normal Form --- 1NF

1NF requires atomic values and no repeating groups.

Good examples:

```
Food_Item
-----
item_id
name
price
```

instead of:

```
Food_Item
-----
item_id
name
prices = "50,60,70"
```

Your Order_Items table correctly stores one item occurrence per row.

15.2 Second Normal Form --- 2NF

2NF eliminates partial dependency on part of a composite key.

Shop_Staff uses:

```
PRIMARY KEY (shop_id, staff_id)
```

Attributes such as:

```
joined_at
```

depend on the combination.

This is a useful 2NF example.

15.3 Third Normal Form --- 3NF

Non-key attributes should depend on the key, the whole key, and nothing but the key.

For example, Area information belongs in Area rather than being repeatedly stored in Shop.

Instead of:

```
Shop
shop_id
shop_name
area_name
area_description
```

you have:

```
Shop
shop_id
area_id
```

and:

```
Area
area_id
name
description
```

This reduces update anomalies.

16. Functional Dependencies to Explain

Examples:

```
user_id → name, email, phone, password_hash
```

```
shop_id → name, owner_id, area_id, status
```

```
item_id → shop_id, category_id, name, price
```

```
order_id → customer_id, shop_id, order_time, status, total_amount
```

For Shop_Staff:

```
(shop_id, staff_id) → joined_at
```

These functional dependencies provide a formal explanation for normalization.

17. Integrity Constraints

CampusBite should demonstrate three major categories of integrity.

Entity Integrity

Primary keys cannot be NULL.

Example:

```
PRIMARY KEY (user_id)
```

Referential Integrity

Foreign keys must refer to valid parent records.

Example:

```
FOREIGN KEY (shop_id)  
REFERENCES Shop(shop_id)
```

This prevents an Order from referring to a non-existent shop.

Domain Integrity

Values must belong to an allowed domain.

Examples:

```
CHECK (price >= 0)
```

```
CHECK (quantity > 0)
```

```
CHECK (rating BETWEEN 1 AND 5)
```

18. Add CHECK Constraints

Recommended:

```
CHECK (price >= 0)
```

```
CHECK (quantity > 0)
```

```
CHECK (total_amount >= 0)
```

```
CHECK (amount >= 0)
```

```
CHECK (open_time <> close_time)
```

and:

```
CHECK (rating BETWEEN 1 AND 5)
```

This demonstrates that validation does not have to exist only in frontend/backend code.

19. Indexing Strategy

Indexes are one of the best DBMS topics to demonstrate.

Useful indexes include:

```
CREATE INDEX idx_shop_area  
ON Shop(area_id);
```

```
CREATE INDEX idx_food_shop_available  
ON Food_Item(shop_id, is_available);
```

```
CREATE INDEX idx_orders_customer_time  
ON Orders(customer_id, order_time);
```

```
CREATE INDEX idx_orders_shop_status  
ON Orders(shop_id, status);
```

```
CREATE INDEX idx_order_items_order  
ON Order_Items(order_id);
```

```
CREATE INDEX idx_history_order_time  
ON Order_Status_History(order_id, changed_at);
```

20. Composite Index Innovation

Do not merely create indexes randomly.

Explain them based on actual query patterns.

Suppose staff frequently runs:

```
SELECT *  
FROM Orders  
WHERE shop_id = ?  
      AND status IN ('placed', 'preparing')  
ORDER BY order_time;
```

Then an index such as:

```
(shop_id, status, order_time)
```

may be useful.

You can demonstrate the difference using:

```
EXPLAIN  
SELECT ...
```

and, where supported by your MySQL version/configuration:

```
EXPLAIN ANALYZE  
SELECT ...
```

This makes the project significantly stronger from a DBMS perspective.

21. Query Optimization Demonstration

Create a section in your project report called:

Before Index

Run:

```
EXPLAIN  
SELECT *  
FROM Orders  
WHERE customer_id = 25  
ORDER BY order_time DESC;
```

Record the execution plan.

Then create:

```
CREATE INDEX idx_customer_order_time  
ON Orders(customer_id, order_time);
```

Run EXPLAIN again.

Discuss:

- Possible key
- Chosen key
- Rows examined
- Access type
- Sorting
- Cost/time

This is much more convincing than simply saying:

"We added indexes for performance."

22. Transactions

Order placement is the strongest transaction example.

A simplified transaction:

```
START TRANSACTION;  
  
INSERT INTO Orders (...);  
  
INSERT INTO Order_Items (...);  
INSERT INTO Order_Items (...);  
  
INSERT INTO Payment (...);  
  
COMMIT;
```

If something goes wrong:

```
ROLLBACK;
```

23. ACID Properties

Atomicity

Either the entire order operation succeeds or it does not.

Consistency

Foreign keys and constraints prevent invalid states.

Isolation

Two users placing orders simultaneously should not corrupt each other's data.

Durability

After COMMIT, the committed transaction should survive normal system failures.

24. Concurrency Control --- A Strong DBMS Demonstration

Consider two customers ordering the last available item.

Without proper concurrency handling:

```
Customer A sees stock = 1
```

```
Customer B sees stock = 1
```

```
A orders
```

```
B orders
```

```
Actual stock becomes negative or inconsistent
```

If you add inventory, use transactions and row locking.

Example concept:

```
SELECT stock_quantity
FROM Inventory
WHERE item_id = ?
FOR UPDATE;
```

Then update the inventory inside the same transaction.

This is an excellent demonstration of:

- Concurrency
- Locking
- Race conditions
- Isolation
- Transaction management

25. Inventory --- Recommended Extension

Add:

```
Inventory
-----
item_id PK/FK
stock_quantity
reorder_level
updated_at
```

But there is an even more interesting model:

```
Inventory_Transaction
-----
inventory_txn_id
item_id
quantity_change
txn_type
reference_id
created_at
```

For example:

```
+20 RESTOCK
-1 SALE
-2 SALE
+10 RESTOCK
```

This provides an auditable inventory ledger rather than only storing a mutable stock number.

26. Innovative Idea #1 --- Smart Queue / ETA Prediction

Instead of showing only:

```
Order #1052 - Preparing
```

calculate an estimated preparation time.

Possible inputs:

- Number of pending orders
- Historical preparation time
- Current shop load
- Item preparation category
- Time of day

A database view could calculate current queue size:

```
SELECT
    shop_id,
    COUNT(*) AS active_orders
FROM Orders
WHERE status IN ('placed', 'preparing')
GROUP BY shop_id;
```

Then the application can combine this with historical statistics.

This makes the project feel like a real campus platform rather than a CRUD application.

27. Innovative Idea #2 --- Historical Menu Pricing

Instead of only:

```
Food_Item.price
```

create:

```
Food_Item_Price_History
-----
price_history_id
item_id
price
valid_from
valid_to
changed_by
```

Now you can answer:

What did this item cost on September 10?

This introduces **temporal data modeling**.

Example:

Burger

₹70 Jan 1 - Mar 31

₹75 Apr 1 - Jun 30

₹80 Jul 1 - current

This is an excellent advanced DBMS concept.

28. Innovative Idea #3 --- Order State Machine

Do not allow arbitrary status changes.

Define valid transitions:

PLACED → PREPARING

PLACED → CANCELLED

PREPARING → READY

PREPARING → CANCELLED

READY → COMPLETED

COMPLETED → no normal transition

CANCELLED → no normal transition

A trigger or stored procedure can enforce these transitions.

For example:

```
change_order_status(order_id, new_status, user_id)
```

The procedure checks:

1. Does the order exist?
2. Is the user authorized?
3. Is the transition legal?
4. Update Orders.
5. Insert Order_Status_History.
6. Commit.

This is much better than allowing arbitrary:

```
UPDATE Orders SET status = ...
```

from every part of the application.

29. Innovative Idea #4 --- Audit Trail

Create:

```
Audit_Log
-----
audit_id
user_id
entity_name
entity_id
action
old_value
new_value
created_at
```

Track important operations:

```
PRICE_CHANGED
ITEM_DISABLED
ORDER_STATUS_CHANGED
SHOP_STATUS_CHANGED
STAFF_ADDED
STAFF_REMOVED
```

This demonstrates:

- Triggers
 - Auditability
 - Historical data
 - Accountability
-

30. Innovative Idea #5 --- Database-Driven Analytics

Create views for:

Shop Sales

```
CREATE VIEW Shop_Sales AS
SELECT
    s.shop_id,
    s.name,
    COUNT(o.order_id) AS total_orders,
    COALESCE(SUM(o.total_amount), 0) AS revenue
FROM Shop s
LEFT JOIN Orders o
    ON s.shop_id = o.shop_id
    AND o.status = 'completed'
GROUP BY s.shop_id, s.name;
```

Then the owner dashboard can simply query:

```
SELECT *
FROM Shop_Sales;
```

This demonstrates **database views**.

31. Innovative Idea #6 --- Top- Selling Items Using Window Functions

Example:

```
SELECT
    shop_id,
    item_id,
    total_quantity,
    RANK() OVER (
        PARTITION BY shop_id
        ORDER BY total_quantity DESC
    ) AS item_rank
FROM ...
```

This demonstrates modern SQL analytics.

You can create reports such as:

```
Shop A
1. Chicken Roll
2. Cold Coffee
3. Veg Sandwich
```

without implementing ranking logic in application code.

32. Innovative Idea #7 --- Personalized Recommendations

CampusBite can recommend items based on order history.

For example:

```
Student frequently orders:
- Coffee
- Sandwich

Recommendation:
"Students who ordered Sandwich also frequently ordered Cold Coffee."
```

A simple SQL-based recommendation system can use co-occurrence.

Conceptually:

```
Order A contains Burger + Coke
Order B contains Burger + Coke
Order C contains Burger + Fries

Burger → Coke: 2
Burger → Fries: 1
```

The database can calculate item associations.

This is an interesting extension because it combines:

- Self-joins
- Aggregation
- Subqueries/CTEs
- Analytics

without requiring machine learning.

33. Innovative Idea #8 --- Peak-Hour Analytics

Calculate:

```
07:00-09:00
09:00-11:00
11:00-13:00
13:00-15:00
...
```

and show:

```
North Square
Peak period: 12:30-14:00
Average orders/hour: 43
```

This can help shop owners plan staffing.

34. Innovative Idea #9 --- Campus Food Availability

Create a campus-level dashboard:

North Square

Open Shops: 4

Closed Shops: 1

Active Orders: 17

Gymkhana

Open Shops: 3

Closed Shops: 0

Active Orders: 8

This requires joins and aggregation across:

Area

→ Shop

→ Orders

35. Innovative Idea #10 --- Database-Based Fraud/Anomaly Detection

For a DBMS project, you can demonstrate simple rules such as:

Unusually many failed payments
Repeated cancelled orders
Multiple orders created within seconds
Unexpected price change

Create an audit/report query rather than claiming this is a production fraud system.

Example:

```
SELECT customer_id, COUNT(*) AS cancelled_orders
FROM Orders
WHERE status = 'cancelled'
GROUP BY customer_id
HAVING COUNT(*) >= 5;
```

This demonstrates:

- GROUP BY
- HAVING
- Aggregate functions

36. Stored Procedures

Good candidates for procedures:

Place Order

```
sp_place_order
```

Responsibilities:

- Validate shop
- Validate items
- Create order
- Insert order items
- Calculate total
- Create payment
- Commit/rollback

Change Order Status

```
sp_change_order_status
```

Responsibilities:

- Validate transition
- Update order
- Write history
- Record user

Add Menu Item

```
sp_add_food_item
```

Generate Shop Sales Report

```
sp_shop_sales_report
```

Stored procedures allow you to demonstrate procedural logic inside the DBMS.

37. Stored Functions

Examples:

```
fn_order_total(order_id)
```

or:

```
fn_shop_rating(shop_id)
```

For example:

```
Average rating =  
SUM(rating) / COUNT(reviews)
```

Use functions selectively. Do not move every piece of business logic into MySQL.

A good architecture divides responsibilities sensibly between backend and DB.

38. Triggers

Good trigger candidates:

Trigger 1 --- Order Status History

Whenever Orders.status changes:

```
INSERT INTO Order_Status_History
```

Trigger 2 --- Price Audit

Whenever Food_Item.price changes:

```
INSERT INTO Food_Item_Price_History
```

Trigger 3 --- Updated Timestamp

Maintain:

```
updated_at
```

automatically.

Trigger Caution

Do not create triggers for every simple validation.

Too many triggers make the system harder to understand.

Use them where **automatic database-side enforcement/history** is genuinely useful.

39. Views

Recommended views:

```
vw_available_menu
vw_shop_sales
vw_customer_order_history
vw_active_orders
vw_shop_rating
vw_daily_shop_revenue
```

Example:

```
CREATE VIEW vw_available_menu AS
SELECT
    fi.item_id,
    fi.shop_id,
    s.name AS shop_name,
    fi.name AS item_name,
    c.name AS category_name,
    fi.price
FROM Food_Item fi
JOIN Shop s
    ON fi.shop_id = s.shop_id
JOIN Category c
    ON fi.category_id = c.category_id
WHERE fi.is_available = TRUE;
```

40. SQL Concepts You Should Demonstrate

Create a dedicated SQL demonstration file containing examples of:

DDL

```
CREATE TABLE
ALTER TABLE
CREATE INDEX
CREATE VIEW
DROP VIEW
```

DML

INSERT
UPDATE
DELETE

DQL

SELECT

TCL

START TRANSACTION
COMMIT
ROLLBACK
SAVEPOINT

DCL

If your deployment/user setup allows it:

GRANT
REVOKE

41. Joins to Demonstrate

Your project naturally supports:

INNER JOIN

Menu:

Food_Item + Category

LEFT JOIN

Shops with zero orders:

```
Shop LEFT JOIN Orders
```

SELF JOIN

Category hierarchy:

```
Category JOIN Category
```

Multiple JOINS

Customer order report:

```
Users  
→ Orders  
→ Order_Items  
→ Food_Item  
→ Shop
```

These should appear in your DBMS demonstration.

42. Subqueries

Example:

```
Find shops whose revenue is greater than the average shop revenue.
```

Conceptually:

```
SELECT ...  
FROM ...  
WHERE revenue >  
(  
    SELECT AVG(revenue)  
    FROM ...  
);
```

This is a useful viva query.

43. CTEs

Example:

```
WITH ShopRevenue AS (  
    SELECT  
        shop_id,  
        SUM(total_amount) AS revenue  
    FROM Orders  
    WHERE status = 'completed'  
    GROUP BY shop_id  
)  
SELECT *  
FROM ShopRevenue  
ORDER BY revenue DESC;
```

CTEs make complex analytical queries easier to understand.

44. Window Functions

Use:

- RANK
- DENSE_RANK
- ROW_NUMBER
- SUM() OVER
- AVG() OVER

Example use:

Rank food items by sales within each shop.

This is an excellent advanced SQL demonstration.

45. Referential Actions

Think carefully about:

```
ON DELETE CASCADE
ON DELETE RESTRICT
ON DELETE SET NULL
```

Example:

Deleting an Order should probably delete its Order_Items because those rows have no meaning without the order.

But deleting a User should **not automatically delete financial/order history**.

This is an excellent place to discuss database lifecycle design.

For important historical records, soft deletion or restricted deletion is often more appropriate.

46. Soft Delete

Instead of:

```
DELETE FROM Shop ...
```

you can add:

```
is_deleted
deleted_at
```

or:

```
status
```

This preserves historical records.

For example:

```
Shop status:
ACTIVE
INACTIVE
SUSPENDED
```

This is especially useful because historical orders should remain meaningful even if a shop stops operating.

47. Payment Design

Current Payment is a good 1:1 design because:

```
Orders.order_id
      ↓
Payment.order_id UNIQUE
```

means one order has at most one Payment record.

However, real payment systems may have multiple attempts.

A more realistic design is:

```
Orders 1 — N Payment_Attempt
```

Example:

```
Attempt 1 → failed
Attempt 2 → failed
Attempt 3 → success
```

Then the database models real payment retries.

For a DBMS project, this is an interesting optional upgrade.

48. Security

Never store plaintext passwords.

Store:

```
password_hash
```

using a modern password hashing algorithm such as Argon2id or bcrypt.

Never execute SQL by string concatenation.

Bad:

```
"SELECT * FROM Users WHERE email = '" + email + "'"
```

Good:

```
SELECT * FROM Users WHERE email = ?
```

with a bound parameter.

This protects against SQL injection.

49. Authorization

Authentication asks:

Who are you?

Authorization asks:

What are you allowed to do?

Example:

Customer

- ✓ Place order
- ✓ View own orders
- ✗ Modify another customer's order

Staff

- ✓ Process orders for assigned shop
- ✗ Process orders from another shop

Owner

- ✓ Manage own shop
- ✗ Modify another owner's shop

This is especially important for CampusBite.

50. Database-Level Security

For a stronger DBMS project, do not let the application connect using the MySQL root account.

Create a dedicated database user:

```
campusbite_app
```

Grant only required privileges.

For example:

```
SELECT
INSERT
UPDATE
DELETE
EXECUTE
```

as appropriate.

This demonstrates **least privilege**.

51. Recommended Additional Tables

A mature version of the schema could contain:

```
Users
Student_Profile
Teacher_Profile

Area

Shop
Shop_Staff
Shop_Timing

Category
Food_Item
Food_Item_Price_History

Orders
Order_Items
Order_Status_History

Payment_Attempt

Review

Inventory
Inventory_Transaction

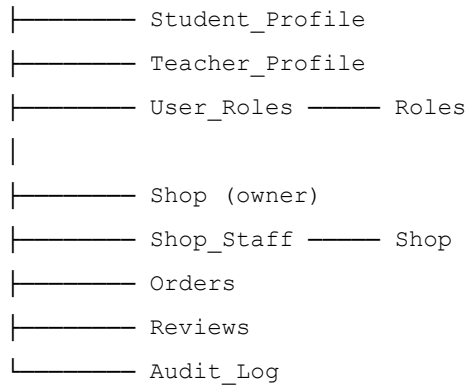
Audit_Log

Roles
User_Roles
```

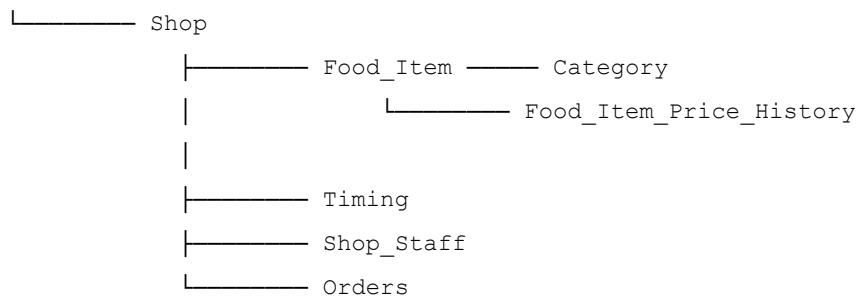
You do not have to implement every table in version 1.

52. Recommended Final Relationship Model

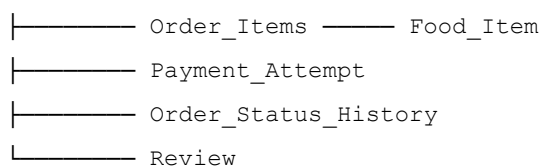
Users



Area



Orders



53. Important Business Rules

Document these explicitly in your project.

BR-01

A shop belongs to exactly one area.

BR-02

A food item belongs to exactly one shop.

BR-03

An order belongs to exactly one customer and one shop.

BR-04

An order must contain at least one Order_Item.

BR-05

An Order_Item quantity must be positive.

BR-06

An order item must belong to the same shop as its order.

This is a particularly important integrity rule.

BR-07

A customer can review only their own completed order.

BR-08

An order should not move backwards through the normal status workflow.

BR-09

Only authorized staff assigned to a shop may process that shop's orders.

BR-10

Historical orders retain the price applicable when they were placed.

BR-11

A payment amount should correspond to the order amount.

BR-12

A food item with `is_available = false` cannot be newly ordered.

BR-13

A user email must be unique.

BR-14

Student registration numbers must be unique.

BR-15

Teacher employee IDs must be unique.

Writing these rules makes the project look substantially more rigorous.

54. Critical Integrity Issue: Order and Item Shop Consistency

Your current schema permits a theoretically invalid state:

```
Order
shop_id = Shop A

Order_Items
item_id = Item belonging to Shop B
```

The foreign key only verifies that the item exists.

It does not automatically verify:

```
Order.shop_id = Food_Item.shop_id
```

This is a great DBMS discussion point.

Possible solutions:

Option A --- Backend validation

Before insertion:

```
SELECT shop_id FROM Food_Item WHERE item_id = ?
```

and compare with order.shop_id.

Option B --- Composite foreign key

Design Food_Item with a suitable unique key and use a composite reference.

Option C --- Stored procedure

Make all order creation go through:

```
sp_place_order
```

which validates the rule.

For an academic project, demonstrate this explicitly.

55. Cart Design

A cart can be implemented in two ways.

Simple approach

Keep the cart in frontend/session state until checkout.

Then only Orders/Order_Items are stored permanently.

This is easy.

Database-backed cart

Create:

```
Cart
----
cart_id
customer_id
created_at
updated_at
```

```
Cart_Item
-----
cart_id
item_id
quantity
```

This demonstrates another M:N relationship and lets users retain carts across devices.

For a DBMS-focused project, the database-backed cart is a useful optional extension.

56. Why an Order Should Belong to One Shop

Do not allow:

```
Order #500
Burger from Shop A
Coffee from Shop B
```

in version 1.

Instead:

```
Order #500 → Shop A
```

and the user creates a separate order for Shop B.

Benefits:

- One pickup location
- One preparation queue
- Simpler payment

- Simpler staff authorization
- Simpler order status
- Simpler transaction model

This significantly simplifies the relational design.

57. Recommended API Structure

```
/api/auth
  POST /register
  POST /login

/api/areas
  GET /

/api/shops
  GET /
  GET /:shopId
  GET /:shopId/menu
  GET /:shopId/timing

/api/orders
  POST /
  GET /my-orders
  GET /:orderId
  PATCH /:orderId/cancel

/api/staff/orders
  GET /
  PATCH /:orderId/status

/api/owner
  POST /items
  PATCH /items/:itemId
  DELETE /items/:itemId
  GET /sales

/api/reviews
  POST /
  GET /shop/:shopId
```

58. Suggested Backend Structure

```
backend/  
├─ src/  
|   ├─ controllers/  
|   ├─ services/  
|   ├─ repositories/  
|   ├─ routes/  
|   ├─ middleware/  
|   ├─ db/  
|   |   ├─ connection.ts  
|   |   └─ migrations/  
|   ├─ sql/  
|   └─ validators/  
|       └─ app.ts  
├─ tests/  
└─ package.json
```

Keep SQL in a dedicated layer so the DBMS logic remains visible.

59. Suggested Frontend Structure

```
frontend/  
├─ src/  
|   ├─ components/  
|   ├─ pages/  
|   ├─ layouts/  
|   ├─ hooks/  
|   ├─ services/  
|   ├─ types/  
|   └─ utils/  
└─ package.json
```

Main screens:

Login
Register
Home
Areas
Shop Listing
Shop Menu
Cart
Checkout
Order Tracking
Order History
Profile
Staff Dashboard
Owner Dashboard
Admin Dashboard

60. Development Workflow

Phase 1 --- Requirements

Document:

- Actors
- Use cases
- Business rules
- Functional requirements
- Non-functional requirements

Phase 2 --- ER Modeling

Create:

Entities
Attributes
Primary keys
Foreign keys
Relationships
Cardinalities
Participation constraints

Phase 3 --- Relational Mapping

Convert ER model to tables.

Document:

```
1:1 → FK/UNIQUE
1:N → FK on N side
M:N → Junction table
Recursive → Self-FK
Specialization → Profile tables
```

Phase 4 --- Normalization

Show:

```
UNF
↓
1NF
↓
2NF
↓
3NF
```

Explain the functional dependencies.

Phase 5 --- Database Implementation

Create:

```
schema.sql
seed.sql
views.sql
procedures.sql
functions.sql
triggers.sql
indexes.sql
```

Phase 6 --- Backend

Implement:

```
Authentication
Authorization
Menu APIs
Order APIs
Payment APIs
Staff APIs
Owner APIs
Review APIs
```

Phase 7 --- Transactions

Implement order creation as a transaction.

Test:

```
Success → COMMIT
Failure → ROLLBACK
```

Phase 8 --- Optimization

Use:

```
EXPLAIN
EXPLAIN ANALYZE
```

before and after indexing.

Phase 9 --- Testing

Test:

- Invalid FK
- Duplicate email
- Negative price

- Quantity = 0
- Invalid order status
- Unauthorized staff
- Wrong shop item
- Failed payment
- Concurrent order
- Review before completion
- Review for someone else's order

61. Testing Strategy

Create a DBMS test matrix.

Test Expected result

Duplicate email Rejected Invalid shop FK Rejected Negative item price Rejected Quantity 0 Rejected Invalid rating
Rejected Invalid order transition Rejected Unauthorized staff update Rejected Wrong-shop item in order Rejected Failed
transaction Rollback Valid order Commit Duplicate review Rejected Deleted referenced area Restricted/rejected
according to policy

This is excellent material for your final report and viva.

62. Sample Advanced Queries

Customers who have spent the most

```
SELECT
    customer_id,
    COUNT(*) AS orders_count,
    SUM(total_amount) AS total_spent
FROM Orders
WHERE status = 'completed'
GROUP BY customer_id
ORDER BY total_spent DESC;
```

Shops with no orders

```
SELECT
    s.shop_id,
    s.name
FROM Shop s
LEFT JOIN Orders o
    ON s.shop_id = o.shop_id
WHERE o.order_id IS NULL;
```

Average shop rating

```
SELECT
    shop_id,
    AVG(rating) AS average_rating,
    COUNT(*) AS review_count
FROM Review
GROUP BY shop_id;
```

Most popular food items

```
SELECT
    oi.item_id,
    fi.name,
    SUM(oi.quantity) AS quantity_sold
FROM Order_Items oi
JOIN Food_Item fi
    ON fi.item_id = oi.item_id
JOIN Orders o
    ON o.order_id = oi.order_id
WHERE o.status = 'completed'
GROUP BY oi.item_id, fi.name
ORDER BY quantity_sold DESC;
```

Daily revenue

```
SELECT
    DATE(order_time) AS order_date,
    SUM(total_amount) AS revenue
FROM Orders
WHERE status = 'completed'
GROUP BY DATE(order_time)
ORDER BY order_date;
```

63. Database Concepts Checklist

Your final project can explicitly claim coverage of:

DBMS concept CampusBite implementation

ER model CampusBite ER diagram Relational model MySQL tables Primary key All major entities Candidate key email, registration number, employee ID Foreign key Shop → Area, Order → Customer, etc. Composite key Shop_Staff UNIQUE email, phone, review/order, etc. NOT NULL Mandatory attributes CHECK price, quantity, rating 1:1 Order → Payment / User → Profile 1:N Shop → Food_Item M:N Order ↔ Food_Item Recursive relationship Category hierarchy Specialization Student/Teacher profiles Normalization 1NF/2NF/3NF Transactions Order placement ACID Order/payment workflow Concurrency Inventory/order locking Indexing Customer/order/menu queries Query optimization EXPLAIN Views Sales/menu reports Stored procedures Order/status operations Stored functions Totals/ratings Triggers Audit/history CTE Analytics Window functions Rankings Aggregation Sales/revenue Subqueries Comparative reports Joins Multi-table reports Referential integrity Foreign keys Domain integrity CHECK Security Least privilege SQL injection protection Parameterized queries Audit trail Audit_Log Temporal data Price/status history

64. What Makes the Project Innovative?

Do not try to impress the evaluator by adding 50 unnecessary tables.

Instead, choose **3--5 meaningful innovations** and implement them well.

A strong combination would be:

Innovation A --- Order State Machine

Database-enforced valid status transitions.

Innovation B --- Temporal Price History

Store historical menu prices.

Innovation C --- Smart Queue Analytics

Calculate current shop load and estimated waiting time.

Innovation D --- SQL Recommendation Engine

Recommend frequently co-ordered items.

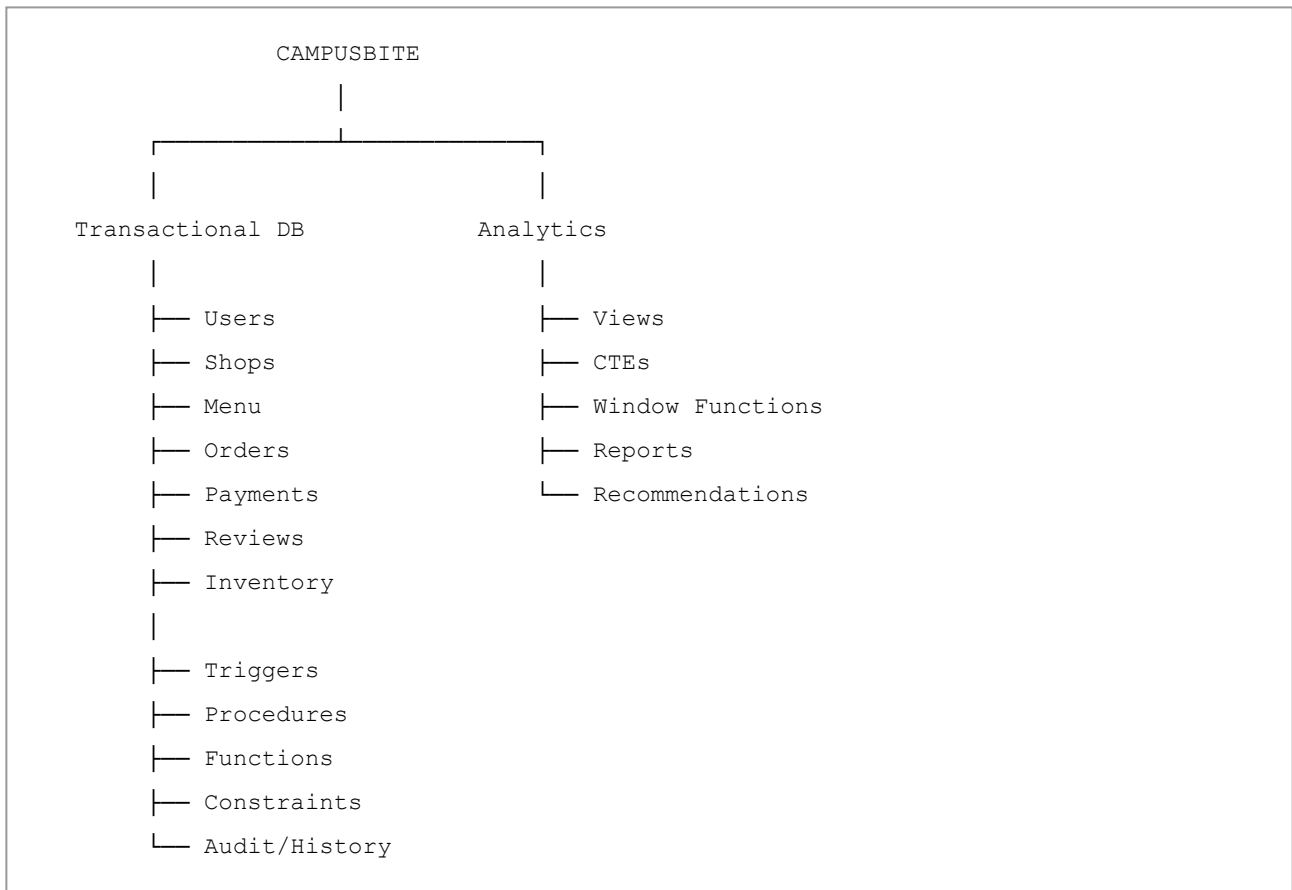
Innovation E --- Audit + Analytics Layer

Use history tables, triggers, views, CTEs and window functions to create an explainable analytics system.

Together, these demonstrate much more DBMS depth than simply adding more CRUD tables.

65. Suggested "Advanced DBMS" Version

If you want CampusBite to be a particularly strong DBMS project, target this architecture:



This clearly separates:

OLTP-style operational data

from:

analytical/reporting queries

while keeping everything understandable for a DBMS course.

66. Final Recommended Schema

A practical final version is:

Users

- |— Student_Profile
- |— Teacher_Profile
- |— User_Roles
 - | └─ Roles
- |
- |— Shop
 - | |— Shop_Staff
 - | |— Timing
 - | └─ Food_Item
 - | └─ Food_Item_Price_History
- |
- |— Orders
 - | |— Order_Items
 - | |— Order_Status_History
 - | |— Payment_Attempt
 - | └─ Review
- |
- |— Audit_Log

Area

- |— Shop

Category

- |— Category

Optional:

Cart

- |— Cart_Item

Inventory

- |— Inventory_Transaction

67. Recommended Project Demonstration

During your final DBMS presentation, do not only show the website.

Show this sequence:

Demo 1 --- ER Diagram

Explain entities and cardinalities.

Demo 2 --- Normalization

Show why the tables are separated.

Demo 3 --- Constraints

Attempt an invalid operation and show that MySQL rejects it.

Demo 4 --- Transaction

Place an order successfully.

Then intentionally cause an error and show rollback.

Demo 5 --- Trigger

Change order status.

Show that Order_Status_History is automatically populated.

Demo 6 --- Stored Procedure

Call:

```
sp_change_order_status(...)
```

and show validation.

Demo 7 --- Indexing

Run EXPLAIN before and after creating an index.

Demo 8 --- Advanced SQL

Show:

- CTE
- Window function
- Subquery
- Aggregation
- Multiple joins

Demo 9 --- Analytics

Show:

Top shops
Top items
Revenue
Peak hours
Average ratings
Active orders

Demo 10 --- Security

Explain:

- Password hashing
- Parameterized queries
- Authentication
- Authorization
- Database least privilege

This makes the DBMS contribution obvious even if the web UI is relatively simple.

68. Recommended Folder Structure for the Whole Project

```
CampusBite/
|
├─ frontend/
|   └─ src/
|       └─ package.json
|
├─ backend/
|   └─ src/
|   └─ tests/
|       └─ package.json
|
├─ database/
|   └─ 01_schema.sql
|   └─ 02_constraints.sql
|   └─ 03_indexes.sql
|   └─ 04_seed.sql
|   └─ 05_views.sql
|   └─ 06_functions.sql
|   └─ 07_procedures.sql
|   └─ 08_triggers.sql
|   └─ 09_analytics.sql
|       └─ migrations/
|
├─ docs/
|   └─ ERD.png
|   └─ relational-schema.md
|   └─ business-rules.md
|   └─ normalization.md
|   └─ API.md
|       └─ DBMS-demo.md
|
├─ docker-compose.yml
├─ README.md
└─ .gitignore
```

69. Final Recommendation

For a DBMS subject, the most important principle is:

Do not make CampusBite just a food-ordering CRUD application. Make the database itself part of the intelligence of the system.

The basic version should provide:

Users
Areas
Shops
Categories
Food Items
Timings
Orders
Order Items
Payments
Reviews

Then strengthen it with:

Constraints
Normalization
Indexes
Transactions
Stored Procedures
Functions
Triggers
Views
Audit History
Temporal Pricing
Concurrency Control
CTEs
Window Functions
Analytics

Finally, add a small number of meaningful innovations:

```
Smart queue estimation
+
Order state machine
+
Historical price tracking
+
SQL-based recommendations
+
Audit/analytics system
```

That combination gives you a project that is simultaneously:

- A usable web application
- A properly normalized relational database
- A demonstration of core DBMS concepts
- A demonstration of advanced SQL
- A demonstration of transaction/concurrency concepts
- A platform for analytics
- A strong candidate for a DBMS viva/project evaluation

The next concrete step should be to **freeze the ER model and revise the SQL schema together**, because the diagram and SQL currently have some differences. In particular, decide whether `Student_Profile`, `Teacher_Profile`, `Review` and `Order_Status_History` are part of the final design, and then make the ERD, relational schema, constraints and SQL implementation exactly consistent.

